Sensors Simulation

Overview

The Panisim simulator provides realistic sensor simulations to enable the development and testing of perception, estimation, and control algorithms. The sensor system is designed to be:

- **Modular**: Each sensor is implemented as a separate class
- **Configurable**: Sensors can be customized through YAML configuration files
- **Realistic**: Includes appropriate noise models and physical placement effects
- **Extensible**: New sensor types can be easily added

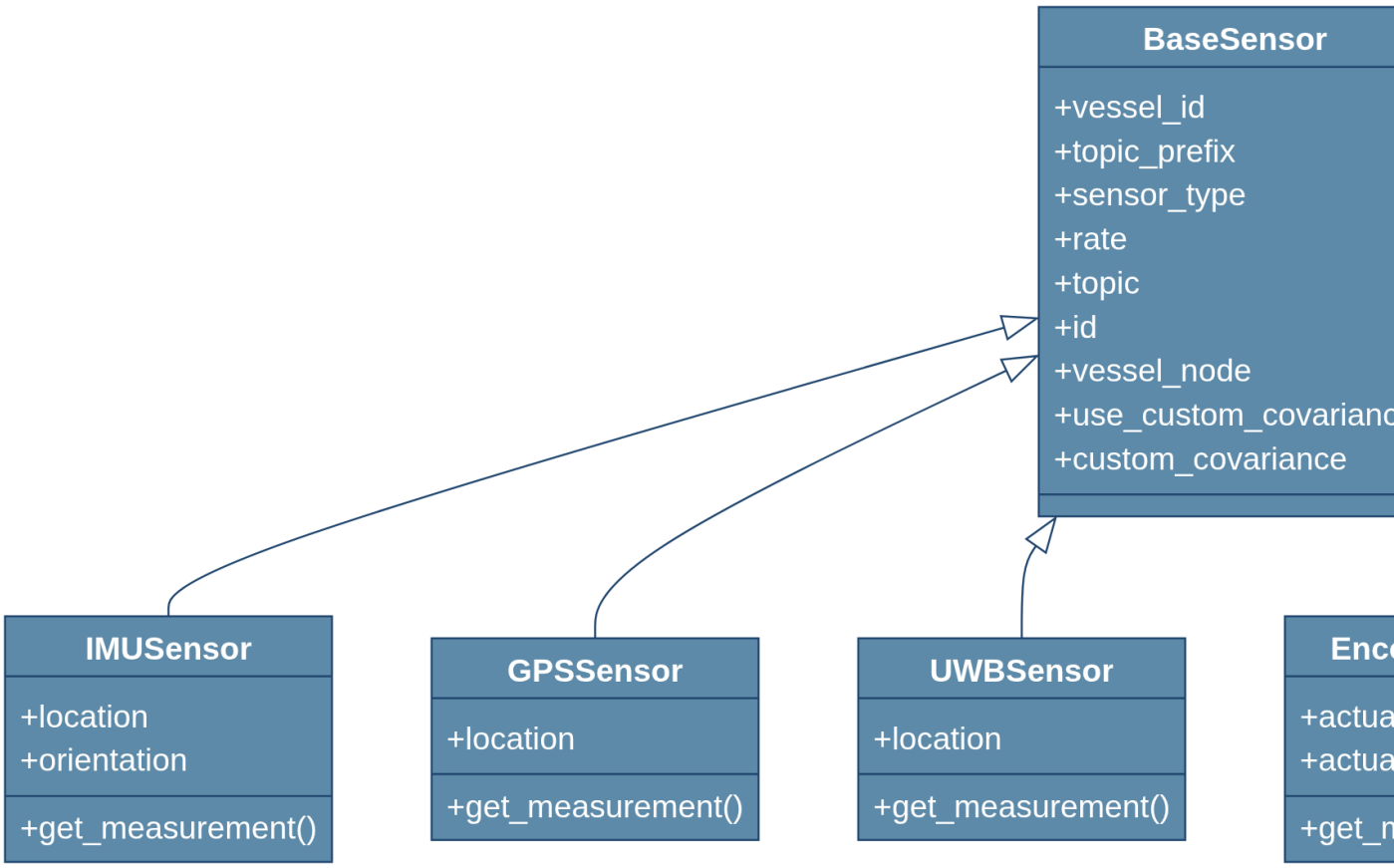
This document explains how sensors are simulated, configured, and how you can create your own custom sensors.

Sensor Architecture

All sensors in the simulator inherit from a common `BaseSensor` class that handles basic functionality such as:

- Sensor identification and topic naming
- Update rate management
- Access to vessel state
- Noise and covariance management

The sensor architecture follows this class hierarchy:



Sensor Types and Simulation

The simulator currently supports the following sensor types:

IMU (Inertial Measurement Unit)

IMU sensors measure: - Orientation (quaternion) - Angular velocity (rad/s) - Linear acceleration (m/s^2)

Simulation details:

The IMU simulation accounts for:

1. Sensor placement relative to the vessel's center of mass
2. Sensor orientation relative to the vessel's body frame
3. Gravitational acceleration effects

4. Centripetal and tangential accelerations due to vessel rotation
5. Realistic noise models for all measurements

Noise model:

- Orientation: Multivariate normal distribution applied to Euler angles
- Angular velocity: Multivariate normal distribution
- Linear acceleration: Multivariate normal distribution

The covariances are defined in the `sensors.yml` file.

```
# Example of how IMU measurements are generated
def get_measurement(self, quat=False):
    state = self.vessel_node.vessel.current_state
    state_der = self.vessel_node.vessel.current_state_der

    # Convert orientation and add noise
    q_sensor = kin.rotm_to_quat(kin.quat_to_rotm(quat) @ kin.quat_to_rotm(orientation_quat))
    q_sensor = kin.eul_to_quat(kin.quat_to_eul(q_sensor) + np.random.multivariate_normal(np.z

    # Calculate acceleration including all effects
    acc_bcs = state_der[0:3] + np.cross(omg_bcs, v_bcs)
    acc_s_bcs = acc_bcs + np.cross(alpha, self.location) + np.cross(omg_bcs, np.cross(omg_bcs
    acc_sensor = kin.quat_to_rotm(orientation_quat).T @ acc_s_bcs

    # Add gravity and noise
    acc_sensor = acc_sensor + kin.quat_to_rotm(q_sensor).T @ np.array([0, 0, -self.vessel_noc
    acc_sensor = acc_sensor + np.random.multivariate_normal(np.zeros(3), self.lin_acc_cov)

    # Calculate angular velocity with noise
    omg_sensor = kin.quat_to_rotm(orientation_quat).T @ omg_bcs
    omg_sensor = omg_sensor + np.random.multivariate_normal(np.zeros(3), self.ang_vel_cov)

    return {...}  # Return measurements and covariances
```

GPS (Global Positioning System)

GPS sensors measure:

- Latitude (degrees)
- Longitude (degrees)
- Altitude (meters)

Simulation details:

The GPS simulation accounts for:

1. Sensor placement on the vessel
2. Conversion between NED (North-East-Down) and LLH (Latitude-Longitude-Height) coordinates
3. Realistic position noise

Noise model:

- Position: Multivariate normal distribution applied to NED coordinates before conversion to LLH

```
# Example of how GPS measurements are generated
def get_measurement(self, quat=False):
    state = self.vessel_node.vessel.current_state
    llh0 = self.vessel_node.vessel.gps_datum

    # Get position in NED, add sensor offset and noise
    ned = state[6:9] + kin.quat_to_rotm(orientation) @ self.location
    ned = ned + np.random.multivariate_normal(np.zeros(3), self.gps_cov)

    # Convert to latitude-longitude-height
    llh = kin.ned_to_llh(ned, llh0)

    return {...}  # Return latitude, longitude, altitude and covariance
```

UWB (Ultra-Wideband Positioning)

UWB sensors measure: - Position in the NED frame (meters)

Simulation details:

The UWB simulation accounts for:

1. Sensor placement on the vessel
2. Position in the NED coordinate frame
3. Realistic position noise (typically higher precision than GPS)

Noise model:

- Position: Multivariate normal distribution applied to NED coordinates

```
# Example of how UWB measurements are generated
def get_measurement(self, quat=False):
    state = self.vessel_node.vessel.current_state
    ned = state[6:9]

    # Get position and add sensor offset and noise
    r_sen = ned + kin.quat_to_rotm(orientation) @ self.location
    r_sen = r_sen + np.random.multivariate_normal(np.zeros(3), self.uwb_cov)

    return {...}  # Return position and covariance
```

Encoder Sensors

Encoder sensors measure:

- Actuator position (degrees for control surfaces, RPM for thrusters)

Simulation details:

The encoder simulation accounts for:

1. Specific actuator type (rudder, thruster, etc.)
2. Unit conversion from state vector to appropriate units
3. Actuator-specific noise characteristics

Noise model:

- Actuator value: Normal distribution with specified RMS noise

```
# Example of how Encoder measurements are generated
def get_measurement(self):
    state = self.vessel_node.vessel.current_state

    # Get the specific actuator value from the state vector
    actuator_value_raw = state[self.state_index]

    # Apply unit conversion (e.g., rad to degrees, rad/s to RPM)
    actuator_value_converted = actuator_value_raw * self.unit_conversion

    # Add noise to the converted measurement
    actuator_value_noisy = actuator_value_converted + np.random.normal(0, self.noise_rms)

    return {...}  # Return actuator value, name, and covariance
```

DVL (Doppler Velocity Log)

DVL sensors measure:

- Linear velocity in the body frame (m/s)

Simulation details:

The DVL simulation accounts for:

1. Sensor placement and orientation
2. Body-frame velocity measurements
3. Realistic velocity noise

Noise model:

- Velocity: Multivariate normal distribution applied to body-frame velocity

```
# Example of how DVL measurements are generated
def get_measurement(self, quat=False):
    state = self.vessel_node.vessel.current_state

    # Get body-frame velocity and add noise
    v_body = state[0:3]
    v_body_noisy = v_body + np.random.multivariate_normal(np.zeros(3), self.vel_cov)

    return {...}  # Return velocity and covariance
```

Sensor Configuration

Sensors are configured through the YAML configuration files. Each sensor definition includes:

- **sensor_type**: The type of sensor (IMU, GPS, UWB, encoder, DVL)
- **sensor_topic**: ROS topic for publishing sensor data
- **sensor_location**: Position of the sensor in the body frame [x, y, z]
- **sensor_orientation**: Orientation of the sensor in the body frame [roll, pitch, yaw]
- **publish_rate**: Update frequency in Hz
- **use_custom_covariance**: Boolean flag to use custom noise parameters
- **custom_covariance**: Custom noise covariance matrices for each measurement

Example configuration for an IMU sensor:

```yaml
sensors:
  -
    sensor_type: IMU
    sensor_topic: /vessel_01/imu/data
    sensor_location: [0.0, 0.0, 0.1]
    sensor_orientation: [0.0, 0.0, 0.0]
    publish_rate: 100
    use_custom_covariance: true
    custom_covariance:
      orientation_covariance: [0.001, 0.0, 0.0, 0.0, 0.001, 0.0, 0.0, 0.0, 0.001]
      angular_velocity_covariance: [0.0001, 0.0, 0.0, 0.0, 0.0001, 0.0, 0.0, 0.0, 0.0001]
      linear_acceleration_covariance: [0.01, 0.0, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0, 0.01]
```

Creating Custom Sensors

To create your own custom sensor, follow these steps:

1. Create a new sensor class

Create a new class that inherits from `BaseSensor` and implements the `get_measurement()` method:

```python
class CustomSensor(BaseSensor):
    def __init__(self, sensor_config, vessel_id, topic_prefix, vessel_node):
        super().__init__(sensor_config, vessel_id, topic_prefix, vessel_node)
        # Initialize sensor-specific properties
        self.location = np.array(sensor_config['sensor_location'])
        self.custom_param = sensor_config.get('custom_param', default_value)

        # Initialize noise parameters
        self.custom_rms = np.array([0.1, 0.1, 0.1])
        self.custom_cov = np.diag(self.custom_rms ** 2)

        # Override with custom covariance if provided
        if self.use_custom_covariance and self.custom_covariance:
            if 'custom_measurement_covariance' in self.custom_covariance:
                self.custom_cov = np.array(self.custom_covariance['custom_measurement_covaria
```

```
    def get_measurement(self, quat=False):
        # Access vessel state
        state = self.vessel_node.vessel.current_state

        # Generate realistic measurements based on state
        # Apply appropriate transformations, physics models, etc.

        # Add noise to measurements
        measurement = calculated_value + np.random.multivariate_normal(np.zeros(3), self.cust

        # Return measurement as a dictionary
        return {
            'custom_measurement': measurement,
            'covariance': self.custom_cov.flatten()
        }
```

2. Register your sensor in the factory function

Modify the `create_sensor` function to include your new sensor type:

```
def create_sensor(sensor_config, vessel_id, topic_prefix, vessel_node):
    """Factory function to create appropriate sensor object based on sensor type"""
    sensor_type = sensor_config['sensor_type']
    sensor_classes = {
        'IMU': IMUSensor,
        'GPS': GPSSensor,
        'UWB': UWBSensor,
        'encoder': EncoderSensor,
        'DVL': DVLSensor,
        'CustomSensor': CustomSensor  # Add your custom sensor here
    }

    if sensor_type not in sensor_classes:
        raise ValueError(f"Unknown sensor type: {sensor_type}")

    return sensor_classes[sensor_type](sensor_config, vessel_id, topic_prefix, vessel_node)
```

3. Create a ROS publisher for your custom sensor

In your vessel's ROS node, add a publisher for your custom sensor:

```python
# In the vessel ROS node initialization
if sensor.sensor_type == 'CustomSensor':
    # Create a publisher for your custom sensor
    # Choose an appropriate message type or create a custom one
    from your_package.msg import CustomSensorMsg

    publisher = self.create_publisher(
        CustomSensorMsg,
        f'{self.topic_prefix}/{sensor.topic}',
        10
    )

    # Add the publisher to the sensor publishers dictionary
    self.sensor_publishers[sensor] = publisher

# In the vessel ROS node update method
if sensor.sensor_type == 'CustomSensor':
    # Create a message for your custom sensor
    msg = CustomSensorMsg()

    # Fill the message with sensor data
    measurement = sensor.get_measurement()
    msg.data = measurement['custom_measurement']
    msg.covariance = measurement['covariance']

    # Add a timestamp
    msg.header.stamp = self.get_clock().now().to_msg()

    # Publish the message
    self.sensor_publishers[sensor].publish(msg)
```

4. Configure your custom sensor in the YAML file

Add your custom sensor to the sensors configuration file:

```yaml
sensors:
  -
    sensor_type: CustomSensor
    sensor_topic: /vessel_01/custom_sensor
    sensor_location: [0.1, 0.0, 0.2]
    sensor_orientation: [0.0, 0.0, 0.0]
```

```yaml
    publish_rate: 50
    custom_param: 42.0
    use_custom_covariance: true
    custom_covariance:
      custom_measurement_covariance: [0.01, 0.0, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0, 0.01]
```

Best Practices for Sensor Simulation

When working with sensors in the simulator, follow these guidelines:

1. **Physical realism**:
   - Place sensors at realistic positions on the vessel
   - Use realistic noise parameters based on actual sensor specifications
   - Consider environmental effects if appropriate (e.g., GPS degradation underwater)
2. **Computational efficiency**:
   - Set appropriate update rates based on real-world sensors
   - Use optimized noise generation methods for high-frequency sensors
3. **Sensor fusion testing**:
   - Configure multiple sensor types to test fusion algorithms
   - Vary noise parameters to test robustness of estimation algorithms
4. **Custom sensors**:
   - Follow the class structure of existing sensors
   - Document physical models and assumptions clearly
   - Use appropriate coordinate transformations
5. **Topic naming**:
   - Use consistent naming conventions for sensors
   - Include vessel ID in topic names for multi-vessel simulations